Interrogation Avancée en SQL : Jointures Relationnelles et Regroupements (GROUP BY & HAVING)

Thème principal : Bases de Données et SQL Avancé

Synthèse exécutive : Ce cours aborde les mécanismes fondamentaux du langage SQL pour croiser des données issues de multiples tables relationnelles via les jointures (INNER JOIN, LEFT JOIN) et structurer les résultats par groupes au moyen des clauses GROUP BY et HAVING. L'objectif est de permettre la construction de requêtes complexes tout en évitant les erreurs classiques de produits cartésiens involontaires ou de filtres incorrects sur données agrégées.

Les Jointures Relationnelles : INNER JOIN vs OUTER JOIN

Dans un SGBDR (Système de Gestion de Base de Données Relationnel), la normalisation impose de séparer les entités dans des tables distinctes. La jointure est l'opération algébrique permettant d'associer ces tables à partir d'un critère d'équivalence. L'évaluation de la clause JOIN s'effectue dans le bloc FROM avant le filtrage WHERE. Il est impératif d'expliciter le critère de liaison dans la clause ON pour éviter un produit cartésien très coûteux en performance.

- INNER JOIN : Restitue uniquement les combinaisons de n-uplets pour lesquelles la condition de jointure est vraie dans les deux tables.
- LEFT JOIN : Conserve la totalité des enregistrements de la table de gauche, complétés par des valeurs NULL si aucune correspondance n'existe à droite.
- Clé Primaire (PK) et Clé Étrangère (FK) : Piliers de l'équijointure reliant les tables entre elles.
- Produit Cartésien (CROSS JOIN) : Résultat d'une jointure sans condition d'association, multipliant chaque ligne de la table A par toutes les lignes de la table B.

Cas concret enseignant : *Le professeur compare la jointure interne (INNER JOIN) à l'accès d'un bâtiment sécurisé : seuls les étudiants possédant une carte valide et une inscription effective entrent. Pour le LEFT JOIN, l'analogie est la liste d'appel globale de l'IUT de Bayonne : tous les étudiants inscrits apparaissent, même si certains n'ont pas encore de note (qui affichera alors 'NULL').*

Agrégation de Données et Regroupement : GROUP BY & HAVING

L'agrégation permet de réaliser des opérations statistiques sur des partitions de données. Dès qu'un GROUP BY est présent, toute colonne du SELECT doit soit figurer dans le GROUP BY, soit être passée dans une fonction d'agrégation. Comme la clause WHERE s'exécute avant le regroupement, elle ne peut pas filtrer un résultat d'agrégation, ce qui justifie l'utilisation de la clause HAVING.

- Fonctions d'Agrégation : Fonctions calculant une valeur unique à partir d'un ensemble de n-uplets (COUNT, SUM, AVG, MIN, MAX).
- GROUP BY : Clause subdivisant les lignes en sous-groupes homogènes partageant les mêmes valeurs sur les attributs spécifiés.
- HAVING : Prédicat de filtrage s'appliquant spécifiquement sur les groupes créés et les résultats des fonctions d'agrégation.
- Ordre logique d'exécution SQL : FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY.

Cas concret enseignant : *L'enseignant illustre le GROUP BY en prenant le tri physique des copies d'examen : on regroupe d'abord les copies par département (GROUP BY), puis pour chaque paquet on calcule la moyenne (AVG). Si l'on souhaite garder uniquement les départements ayant plus de 12 de moyenne, c'est la clause HAVING qui élimine les paquets non conformes.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi est-il impossible d'écrire WHERE AVG(note) > 10 dans une requête SQL ?
Réponse du professeur : Parce que la clause WHERE est évaluée avant le regroupement (GROUP BY), à un moment où les moyennes par groupe n'ont pas encore été calculées. Le filtre sur les agrégats relève exclusivement de la clause HAVING.

Question étudiant : Que se passe-t-il si j'oublie d'inclure une colonne du SELECT dans le GROUP BY ?
Réponse du professeur : Le SGBDR renvoie une erreur car il ne sait pas quelle valeur afficher pour cette colonne parmi toutes celles contenues dans le groupe. Toutes les colonnes non agrégées du SELECT doivent obligatoirement être présentes dans le GROUP BY.

■ Exercice & Démonstration d'Application

Énoncé : Soit la base de données de l'IUT composée des tables Etudiant(id_etud, nom, id_promo) et Note(id_note, id_etud, matiere, note). Écrire la requête SQL qui affiche le nom de l'étudiant et sa moyenne dans la matière 'R1.05', uniquement pour les étudiants ayant au moins 2 notes dans cette matière et une moyenne stricte supérieure à 12, triés par moyenne décroissante.

Résolution pas à pas :
1. Identifier les tables nécessaires : Etudiant (E) et Note (N). 2. Réaliser l'équijointure via E.id_etud = N.id_etud. 3. Filtrer les notes de la matière 'R1.05' avec la clause WHERE N.matiere = 'R1.05'. 4. Regrouper les résultats par identifiant et nom d'étudiant avec GROUP BY E.id_etud, E.nom. 5. Appliquer les filtres sur les groupes avec HAVING COUNT(N.id_note) >= 2 AND AVG(N.note) > 12. 6. Projeter les colonnes E.nom et AVG(N.note) AS moyenne. 7. Trier par ordre décroissant avec ORDER BY moyenne DESC.

Résultat : SELECT E.nom, AVG(N.note) AS moyenne FROM Etudiant E INNER JOIN Note N ON E.id_etud = N.id_etud WHERE N.matiere = 'R1.05' GROUP BY E.id_etud, E.nom HAVING COUNT(N.id_note) >= 2 AND AVG(N.note) > 12 ORDER BY moyenne DESC;

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'application entre la clause WHERE et la clause HAVING ?

R : La clause WHERE filtre les lignes individuelles avant le regroupement, tandis que la clause HAVING filtre les groupes formés après le GROUP BY en s'appuyant sur les fonctions d'agrégation.

Q : Quel est le résultat d'un LEFT JOIN si aucune ligne de la table de droite ne correspond à un enregistrement de la table de gauche ?

R : Toutes les lignes de la table de gauche sont conservées, et les colonnes issues de la table de droite sont complétées par des valeurs NULL.

Q : Si une table A contient 10 lignes et une table B contient 5 lignes, combien de lignes retourne un produit cartésien (CROSS JOIN) entre A et B ?

*R : Il retourne 10 * 5 = 50 lignes.*